Practical NO.06(A)

Name - Sakshi Vijaykumar Muthiyan

Roll No. -

Title - Implement Reinforcement Learning using an example of a maze environment that the agent needs to explore.

In [1]:
```
import numpy as np
import random
import matplotlib.pyplot as plt
# defining a simple 4x4 maze grid layout
# 0 = free cell, -1 = trap/wall, 10 = final exit goal destination
maze = np.array([
    [ 0,  0,  0, -1],
    [-1,  0, -1,  0],
    [ 0,  0,  0, -1],
    [-1, -1,  0, 10]
])
# actions: 0 = Up, 1 = Down, 2 = Left, 3 = Right
actions = [0, 1, 2, 3]
print("=== Maze Layout Matrix ===")
print(maze)
```

```
=== Maze Layout Matrix ===
[[ 0  0  0 -1]
 [-1  0 -1  0]
 [ 0  0  0 -1]
 [-1 -1  0 10]]
```

In [2]:
```
# rows represent cells (4x4 = 16 states) and columns represent the 4 actions
q_table = np.zeros((16, 4))
# learning rate, discount factor, and exploration profile parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
print("Initial Q-Table shape dimensions:", q_table.shape)
print("Preview of empty training matrix weights:")
print(q_table[:4, :]) # printing first 4 rows to show starting state matrix valu
```

```
Initial Q-Table shape dimensions: (16, 4)
Preview of empty training matrix weights:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

In [3]:
```
# helper function to convert 2D coordinates into a single 1D state index number
def get_state_index(row, col):
    return row * 4 + col
print("=== Commencing Reinforcement Learning Training Phase ===")
# training loop simulation across 500 complete episodes
for episode in range(1, 501):
    # agent always resets at top-left corner index (0,0)
    row, col = 0, 0
    while maze[row, col] != 10:
        state = get_state_index(row, col)
        # epsilon-greedy strategy selection check
        if random.uniform(0, 1) < epsilon:
```

```
            action = random.choice(actions)
        else:
            action = np.argmax(q_table[state])
        # compute coordinate adjustments based on action selection index
        next_row, next_col = row, col
        if action == 0 and row > 0: next_row -= 1   # Move Up
        if action == 1 and row < 3: next_row += 1   # Move Down
        if action == 2 and col > 0: next_col -= 1   # Move Left
        if action == 3 and col < 3: next_col += 1   # Move Right
        # checking boundaries and fetching reward points
        reward = maze[next_row, next_col]
        if next_row == row and next_col == col:
            reward = -1 # penalty score if hitting boundary walls
        next_state = get_state_index(next_row, next_col)
        # core bellman equation update step
        old_value = q_table[state, action]
        next_max = np.max(q_table[next_state])
        q_table[state, action] = old_value + alpha * (reward + gamma * next_max
        # update coordinate tracking references
        if maze[next_row, next_col] != -1:
            row, col = next_row, next_col
    # printing training checkpoint validation logs
    if episode % 100 == 0:
        print(f"Episode {episode}/500: Training iteration updates registered suc
```

```
=== Commencing Reinforcement Learning Training Phase ===
Episode 100/500: Training iteration updates registered successfully.
Episode 200/500: Training iteration updates registered successfully.
Episode 300/500: Training iteration updates registered successfully.
Episode 400/500: Training iteration updates registered successfully.
Episode 500/500: Training iteration updates registered successfully.
```

In [5]:
```
import pandas as pd
# displaying the full computed matrix values
print("=== Final Trained Q-Table Weight Matrix ===")
print("Columns layout configuration represents [Up, Down, Left, Right]\n")
# formatting float prints to look clean and neat
pd.set_option('display.float_format', lambda x: '%.2f' % x)
q_df = pd.DataFrame(q_table, columns=['Up', 'Down', 'Left', 'Right'])
print(q_df)
```

```
=== Final Trained Q-Table Weight Matrix ===
Columns layout configuration represents [Up, Down, Left, Right]

      Up  Down  Left  Right
0   3.31 -0.89  0.90   5.90
1   3.88  6.56  3.03   1.38
2  -0.27 -0.47  3.82  -0.41
3   0.00  0.00  0.00   0.00
4   0.00  0.00  0.00   0.00
5   4.73  7.29 -1.00  -1.00
6   0.00  0.00  0.00   0.00
7   0.00  0.00  0.00   0.00
8  -0.27 -0.19  0.25   6.03
9   4.69 -0.65  3.35   8.10
10 -0.89  9.00  5.04  -0.75
11  0.00  0.00  0.00   0.00
12  0.00  0.00  0.00   0.00
13  0.00  0.00  0.00   0.00
14  6.81  5.97 -0.69  10.00
15  0.00  0.00  0.00   0.00
```

In [6]:

```python
print("=== Testing Agent Policy Escape Path ===")
row, col = 0, 0
path_history = [(row, col)]
steps = 0
# trace the path taken by using the maximum value positions learned inside the Q
while maze[row, col] != 10 and steps < 10:
    state = get_state_index(row, col)
    action = np.argmax(q_table[state])
    if action == 0 and row > 0: row -= 1
    elif action == 1 and row < 3: row += 1
    elif action == 2 and col > 0: col -= 1
    elif action == 3 and col < 3: col += 1
    path_history.append((row, col))
    steps += 1
print("Successful exit path route coordinates:", path_history)
# building a simple text layout visual map
visual_map = np.full((4,4), '.')
for r, c in path_history:
    visual_map[r, c] = '*'
visual_map[3, 3] = 'E' # E = Exit Point
print("\n=== Visual Path Map Traversed (*) ===")
for r in visual_map:
    print(" ".join(r))
```

```
=== Testing Agent Policy Escape Path ===
Successful exit path route coordinates: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2),
(3, 2), (3, 3)]

=== Visual Path Map Traversed (*) ===
* * . .
. * . .
. * * .
. . * E
```